

A.4.3 实验内容及步骤

首先要掌握 MyCache 模拟器的使用方法（见 A.4.4 节）。

A.4.3.1 Cache 容量对不命中率的影响

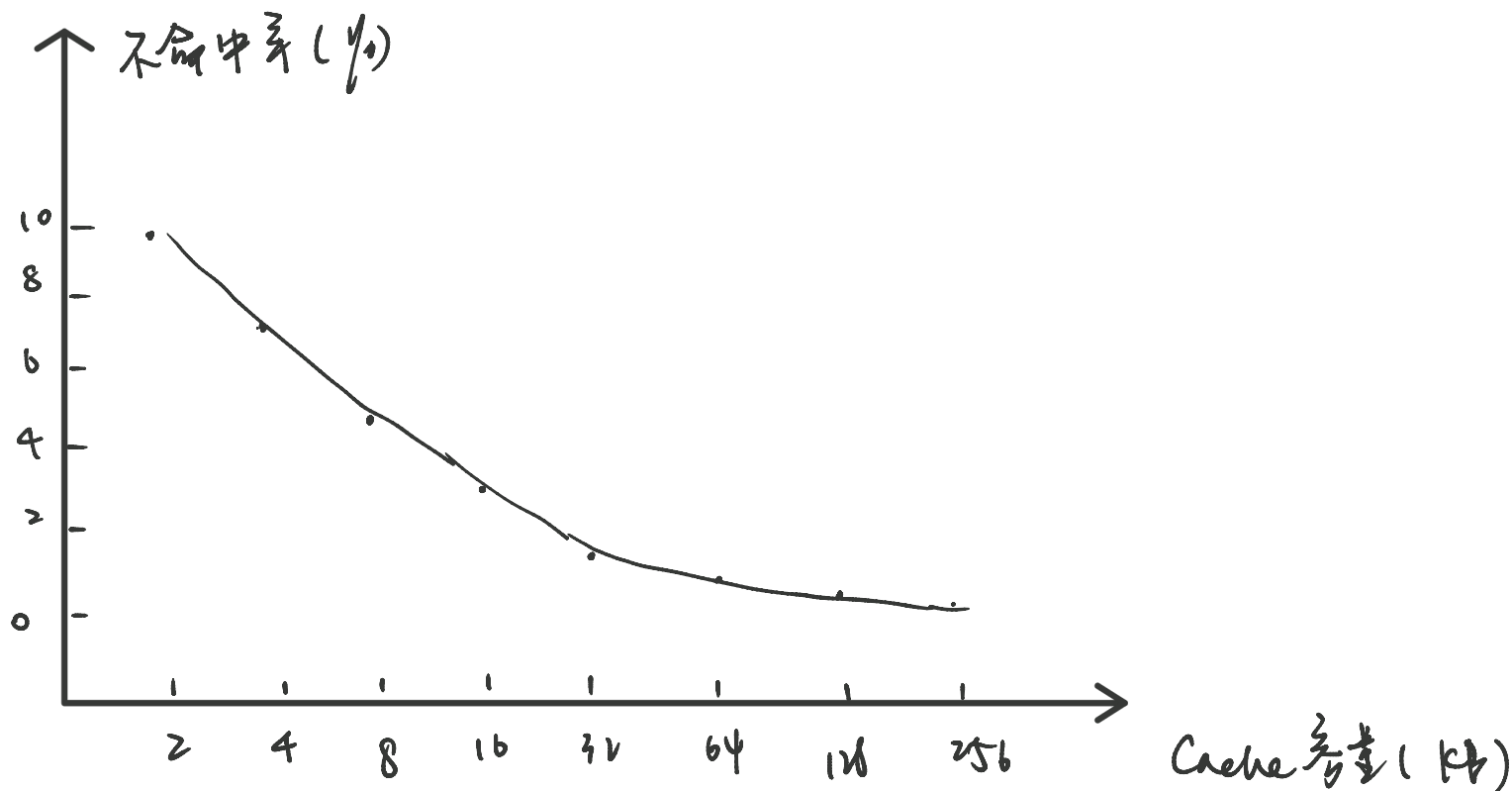
1. 启动 MyCache。
2. 用鼠标点击“复位”按钮，把各参数设置为默认值。
3. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后点击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
4. 选择不同的 **Cache** 容量，包括：2KB, 4KB, 8KB, 16KB, 32KB, 64KB, 128KB, 256KB，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.1 中记录各种情况下的不命中率。

表 A.4.1 不同容量下 Cache 的不命中率

Cache 容量 (KB)	2	4	8	16	32	64	128	256
不命中率	9.87%	7.19%	4.48%	2.65%	1.42%	0.89%	0.60%	0.49%

地址流文件名： all.dsn

5. 以容量为横坐标，画出不命中率随 Cache 容量变化而变化的曲线。并指明地址流文件名。



6. 根据该模拟结果，你能得出什么结论？

在其他多数因素的情况下，随着Cache容量的扩大，不命中率会逐渐降低。因为容量大能容纳更多的数据，减少了结构性冲突和容量失效。但当容量大到一定程度后，不命中率的下降趋势会变缓。

A.4.3.2 相联度对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。此时的Cache容量为64KB。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。

3. 选择不同的 Cache 相联度, 包括: 直接映象, 2 路, 4 路, 8 路, 16 路, 32 路, 分别执行模拟器 (单击“执行到底”按钮即可执行), 然后在表 A.4.2 中记录各种情况下的不命中率。

表 A.4.2 当容量为 64KB 时, 不同相联度下 Cache 的不命中率

相联度	1	2	4	8	16	32
不命中率	1.97%	1.15%	0.99%	0.93%	0.92%	0.91%

地址流文件名: cu.dtu.

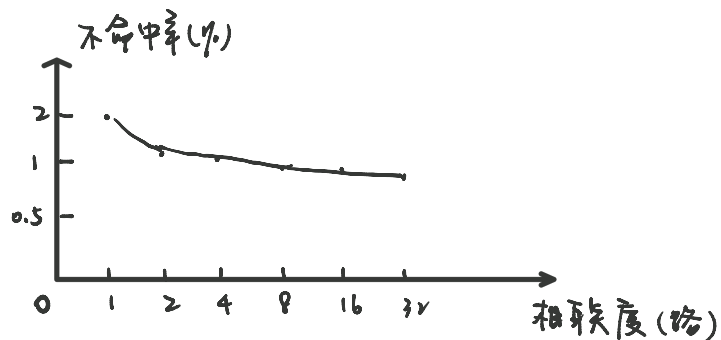
4. 把 Cache 的容量设置为 256KB, 重复(3)的工作, 并填写表 A.4.3。

表 A.4.3 当容量为 256KB 时, 不同相联度下 Cache 的不命中率

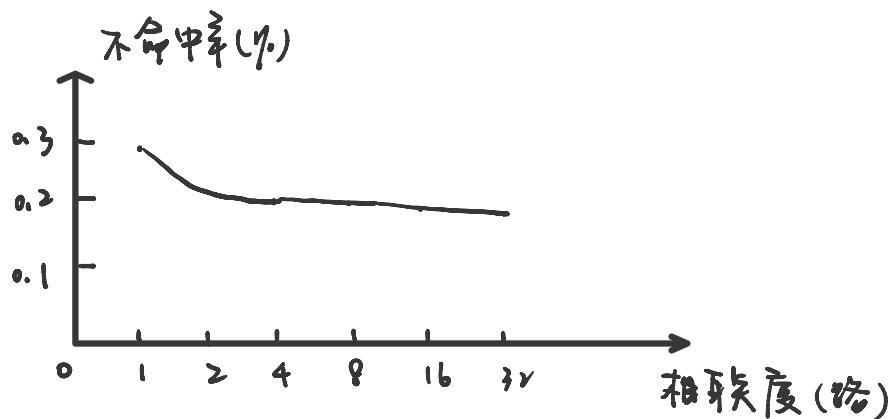
相联度	1	2	4		8	16	32
不命中率	0.30%	0.21%	0.21%		0.21%	0.21%	0.21%

地址流文件名: eg.dtu.

5. 以相联度为横坐标, 画出在 64KB 和 256KB 的情况下不命中率随 Cache 相联度变化而变化的曲线。并指明地址流文件名。



64kb下不命中率随 Cache 相联度变化的曲线



256kb下不命中率随 Cache 相联度变化的曲线

6. 根据该模拟结果，你能得出什么结论？

提高相联度可以有效降低冲突失效，从而降低不命中率。从直接映象(1)到2路、4路时，不命中率下降最明显。当相联度提高到一定程度后，不命中率基本不再下降。此外，容量较大的Cache整体不命中率普遍低于容量较小的Cache。

A.4.3.3 Cache 块大小对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
3. 选择不同的Cache块大小，包括：16B，32B，64B，128B，256B，对于Cache的各种容量，包括：2KB，8KB，32KB，128KB，512KB，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表A.4.4中记录各种情况下的不命中率。

表 A.4.4 各种块大小情况下 Cache 的不命中率 (%)

块大小 (B)	Cache 容量 (KB)				
	2	8	32	128	512
16	12.79	5.74	1.14	0.52	0.42
32	10.88	4.62	0.99	0.32	0.24
64	10.41	4.35	0.89	0.22	0.15
128	11.36	4.50	0.78	0.17	0.10
256	13.73	5.15	0.99	0.15	0.06

地址流文件名: spice.dtu

4. 分析 Cache 块大小对不命中率的影响。

Cache块大小对不命中率的影响呈U形曲线。块大小较小时，增加块大小可以充分利用访问局部性，让一次调块带入更多后续用到的数据，从而降低不命中率；但当块大小过大时，由于Cache容量有限，总行数变少，导致新调入的块更容易频繁擦除旧的

块，且带入的数据可能用不上，从而导致命中率反而上升。

A.4.3.4 替换算法对不命中率的影响

1. 用鼠标单击“复位”按钮，把各参数设置为默认值。
2. 选择一个地址流文件。方法：选择“访问地址”→“地址流文件”选项，然后单击“浏览”按钮，从本模拟器所在的文件夹下的“地址流”文件夹中选取。
3. 对于不同的替换算法、Cache 容量和相联度，分别执行模拟器（单击“执行到底”按钮即可执行），然后在表 A.4.5 中记录各种情况下的不命中率。

表 A.4.5 LRU 和随机替换法的不命中率的比较 (1/0)

Cache 容量	相 联 度					
	2 路		4 路		8 路	
	LRU	随机算法	LRU	随机算法	LRU	随机算法
16KB	0.18	0.33	0.17	0.37	0.19	0.20
64KB	0.15	0.18	0.15	0.15	0.15	0.15
256KB	0.15	0.15	0.15	0.15	0.15	0.15
1MB	0.15	0.15	0.15	0.15	0.15	0.15

地址流文件名: tex.drn

4. 分析不同的替换算法对 Cache 不命中率的影响。

在绝大多数情况下，LRU 的不命中率低于随机替换算法。因为 LRU 利用了时间局部性，保留活跃数据；随机算法没有考虑数据访问的历史规律，容易误删即将被频繁访问的块。

且，当 Cache 容量到达一定瓶颈后，盲目提升硬件复杂度和优化替换算法对降低不命中率不再具有效益。

第五章

5.1

不命中率: CPU访问存储系统时在M1中找不到所需信息的概率。

不命中开销: 从向M2发出访问请求到把整个数据块调入M1中所需的时间。

直接映像: 主存中的每一块只能被放置到Cache中唯一的-个位置。

LRU (最近最少使用法): 选择近期最少被访问的块作为被替换的块。

写回法: 执行“写”操作时, 只写入Cache, 仅当Cache中相应的块被替换时才写回主存。

5.2 Cache-主存层解决的是速度矛盾, 用SRAM自动管, 对软件完全透明。设命中时CPU只挂起等待几个周期。

不切换进程, 用的是主存物理地址。

主存-辅存层解决容量矛盾, 由操作系统和硬件共同管理, 属于虚拟存储。设命中时必须触发中断把当前进程挂起, 切换去执行别的进程, 使用虚拟地址。

5.4

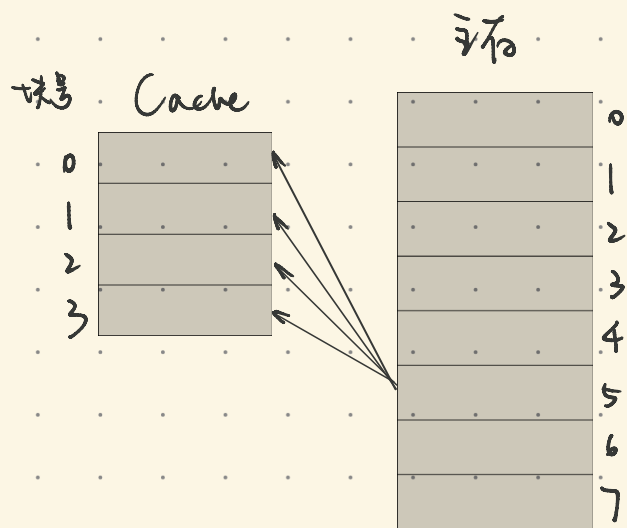
5.4 设有一个“Cache-主存”层次, Cache为4块, 主存为8块。试分别对于以下3种情况, 画出其映像关系示意图, 并计算访存块地址为5时的索引。

(1) 全相联。

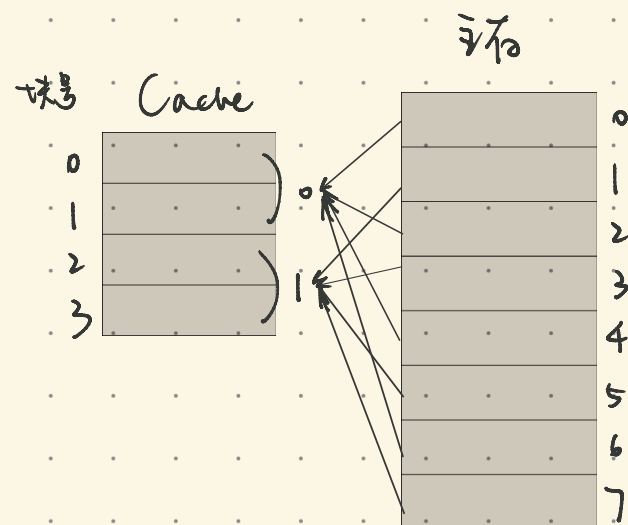
(2) 组相联, 每组两块。

(3) 直接映像。

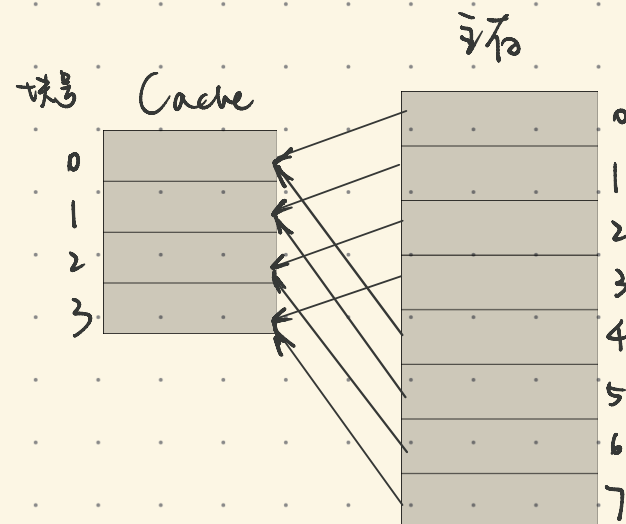
11) 全相联.



12) 组相联.



13) 直接映像.



可能出现在 Cache 块中的
序号-4.

索引 = $5 \bmod 2 = 1$. 第1组的2或3.

索引 = $5 \bmod 4 = 1$. 第1块.

5.10

5.10 给定以下的假设, 试计算直接映像 Cache 和 2 路组相联 Cache 的平均访问时间以及 CPU 的性能。由计算结果能得出什么结论?

- (1) 理想 Cache 情况下的 CPI 为 2.0, 时钟周期为 2 ns, 平均每条指令访存 1.2 次。
- (2) 两种 Cache 容量均为 64 KB, 块大小都是 32 B。
- (3) 组相联 Cache 中的多路选择器使 CPU 的时钟周期增加了 10%。
- (4) 这两种 Cache 的不命中开销都是 80 ns。
- (5) 命中时间为 1 个时钟周期。
- (6) 64 KB 直接映像 Cache 的不命中率为 1.4%, 64 KB 2 路组相联 Cache 的不命中率为 1.0%。

11) 平均访问时间.

直接映像: $AMAT_1 = 2 + 80 \times 1.4\% = 3.12 \text{ ns}$.

2路组相联: $AMAT_2 = 2.2 + 80 \times 1.0\% = 3.00 \text{ ns}$.

12) CPU性能

每条指令时间 = 理想CPI $\times$ 时钟周期 + 平均每条指令访存次数 $\times$ 不命中率 $\times$ 不命中开销.

直接映像: $t_1 = 2.0 \times 2 + 1.2 \times 1.4\% \times 80 = 5.344 \text{ ns}$

2路组相联: $t_2 = 2.0 \times 2.2 + 1.2 \times 1.0\% \times 80 = 5.360 \text{ ns}$

13) 平均访问时间低, 并不意味着整个CPU的实际性能好. 因此, 结构设计必须进行全局折中. 整体评估其对时钟周期和总执行时间的综合影响. 在本题的给定参数假设下, 选择直接映像更明智.